XOR, NOR, <<, >>

bitwise XOR (^), bitwise NOT (~), left shift (<<), right shift (>>). explain me. 2
in. 4
in c prog. 5

bitwise XOR (^), bitwise NOT (~), left shift (<<), right shift (>>). explain me

Sure, Vasu! Let's go over each of these **bitwise operators** one by one, with **simple explanations** and **examples in binary and decimal**.

---

1. Bitwise XOR (`^`)

"XOR" stands for **exclusive OR**. It returns:

- `1` if the bits are **different**
- `0` if the bits are **same**

Example:

```java
int a = 5;   // Binary: 0101
int b = 3;   // Binary: 0011
int result = a ^ b;  // result = 6
```

```markdown
  0101  (5)
^ 0011  (3)
-------
  0110  (6)
```

---

2. Bitwise NOT (`~`)

This is a **unary** operator (works on one operand).
It **inverts** all the bits: `0` becomes `1`, and `1` becomes `0`.

Example:

```java
int a = 5;   // Binary: 0000 0101
int result = ~a;  // result = -6
```

```yaml
~0000 0101 = 1111 1010  (which is -6 in 2's complement)
```

Note: In Java, integers are 32 bits, so the result of `~` is often negative due to the sign bit.

---

3. Left Shift (`<<`)

Shifts the bits **to the left**, and fills **0** from the right.
It's like **multiplying by 2** each time.

Example:

```java
int a = 5;  // Binary: 0000 0101
int result = a << 1;  // result = 10
```

```yaml
0000 0101 << 1 = 0000 1010 (10)
```

Summary:

- **`a << 1`** → a * 2
- **`a << 2`** → a * 4
- **`a << n`** → a * (2^n)

---

4. Right Shift (`>>`)

Shifts the bits **to the right**, and fills **sign bit** from the left.
It's like **dividing by 2** each time.

Example:

```java
int a = 5;  // Binary: 0000 0101
int result = a >> 1;  // result = 2
```

```yaml
0000 0101 >> 1 = 0000 0010 (2)
```

Summary:

- **`a >> 1`** → a / 2
- **`a >> 2`** → a / 4
- **`a >> n`** → a / (2^n)

  If the number is **negative**, the sign bit (1) is used to fill on the left.

Let me know if you'd like to try some practice questions or need help visualizing any of these!

in

It looks like your message got cut off — could you please clarify what you meant by **"in"**?

Are you asking:

- How these operators are **used in Java**?
- What their **effect is on `int` values**?
- How they behave **in specific examples**?
- Or something else?

Let me know so I can help you better!

in c prog

Thanks for the clarification! Here's a full explanation of **bitwise XOR (^), bitwise NOT (~), left shift (<<), and right shift (>>) in C programming** — with examples:

🧠 Basics in C

C supports bitwise operators that work **directly on the binary representation** of integers.

All of these operators work on `int`, `unsigned int`, `char`, etc.

---

1. Bitwise XOR (`^`)

- Returns `1` if bits are different, `0` if same.

```c
#include <stdio.h>
int main() {
    int a = 5;    // 0101
    int b = 3;    // 0011
    int result = a ^ b;  // 0110 → 6
    printf("a ^ b = %d\n", result);
    return 0;
}
```

---

2. Bitwise NOT (`~`)

- Inverts every bit.
- For signed `int`, result may look negative due to **two's complement**.

```c
#include <stdio.h>
int main() {
    int a = 5;        // 0000 0101
    int result = ~a;  // 1111 1010 → -6 (in 32-bit signed int)
    printf("~a = %d\n", result);
    return 0;
}
```

---

3. Left Shift (`<<`)

- Shifts bits to the **left**, fills 0s on the right.
- Equivalent to multiplying by 2^n.

```c
#include <stdio.h>
int main() {
    int a = 5;              // 0000 0101
    int result = a << 1;    // 0000 1010 → 10
    printf("a << 1 = %d\n", result);
    return 0;
}
```

---

4. Right Shift (`>>`)

- Shifts bits to the **right**.
- For **signed int**, the left side may be filled with sign bit (compiler-dependent).

```c
#include <stdio.h>
int main() {
    int a = 5;              // 0000 0101
    int result = a >> 1;    // 0000 0010 → 2
    printf("a >> 1 = %d\n", result);
    return 0;
}
```

🔪 Bonus: Try All in One

```c
#include <stdio.h>

int main() {
    int a = 5, b = 3;

    printf("a ^ b = %d\n", a ^ b);  // 6
    printf("~a = %d\n", ~a);        // -6
    printf("a << 1 = %d\n", a << 1); // 10
    printf("a >> 1 = %d\n", a >> 1); // 2

    return 0;
}
```

Let me know if you want the **binary output** for each step or need **bitwise operations for `unsigned int`** too!